



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



## **REPORTE DE PRÁCTICA N° 01**

**NOMBRE COMPLETO:** Lechuga Castillo Shareny Ixchel

**N° de Cuenta:** 319004252

**GRUPO DE LABORATORIO:** 13

**GRUPO DE TEORÍA:** 04

**SEMESTRE** 2026-1

**FECHA DE ENTREGA LÍMITE:** 28-08-2025

**CALIFICACIÓN:** \_\_\_\_\_

## REPORTE DE PRÁCTICA:

### Actividades realizadas.

- 1.- Ventana cambia el color de fondo de forma random tomando rango de colores RGB y con una periodicidad de 2 segundos.
- 2.- 3 letras iniciales de sus nombres creadas a partir de triángulos, todas las letras son del mismo color.

El código es el siguiente:

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <glew.h>
4  #include <glfw3.h>
5  #include <cstdlib>  //-- Se agregó para color aleatorio
6  #include <ctime>    //-- Se agregó para el tiempo
7  //Dimensiones de la ventana
8  const int WIDTH = 800, HEIGHT = 800;
9  GLuint VAO, VBO, shader;
10
11 //LENGUAJE DE SHADER (SOMBRA) GLSL
12 //Vertex Shader
13 //recibir color, salida Vcolor
14 static const char* vShader = "
15 #version 330
16 layout (location =0) in vec3 pos;
17 void main()
18 {
19     gl_Position=vec4(pos.x,pos.y,pos.z,1.0f);
20 }";
21
22 //Fragment Shader
23 //recibir Vcolor y dar de salida color
24 static const char* fShader = "
25 #version 330
26 out vec4 color;
27 void main()
28 {
29     color = vec4(0.0f,0.0f,0.0f,1.0f);
30 }";
31
32
33
34 void CrearTriangulo()
35 {
36     GLfloat vertices[] = {
37         //----- LETRA S -----
38         //Superior
39         -0.97f, 0.8f,0.0f,
40         -0.45f,0.8f, 0.0f,
41         -0.97f,0.95f,0.0f,
42
43         -0.97f, 0.95f,0.0f,
44         -0.45f,0.95f, 0.0f,
45         -0.45f,0.8f,0.0f,
46
47         //Vertical superior
48         -0.97f, 0.95f,0.0f,
49         -0.97f,0.55f, 0.0f,
50         -0.82f,0.55f,0.0f,
51
52         -0.82f, 0.55f,0.0f,
53         -0.82f,0.95f, 0.0f,
54         -0.97f,0.95f,0.0f,
55
56         //Medio
57         -0.97f, 0.55f,0.0f,
58         -0.45f,0.55f, 0.0f,
59         -0.97f,0.70f, 0.0f,
60
61         -0.97f, 0.70f,0.0f,
62         -0.45f,0.70f, 0.0f,
63         -0.45f,0.55f, 0.0f,
64
65         //Vertical inferior
66         -0.45f,0.70f, 0.0f,
67         -0.45f,0.30f, 0.0f,
68         -0.60f,0.30f, 0.0f,
```

```

70      -0.60f,0.30f, 0.0f,
71      -0.60f,0.70f, 0.0f,
72      -0.45f,0.70f, 0.0f,
73
74      //Inferior
75      -0.97f,0.30f, 0.0f,
76      -0.45f,0.30f, 0.0f,
77      -0.97f,0.45f, 0.0f,
78
79      -0.97f,0.45f, 0.0f,
80      -0.45f,0.45f, 0.0f,
81      -0.45f,0.30f, 0.0f,
82
83      //----- LETRA L -----
84      //Vertical l
85      -0.35f,0.45f, 0.0f,
86      -0.20f,0.45f, 0.0f,
87      -0.35f,-0.25f, 0.0f,
88
89      -0.35f,-0.25f, 0.0f,
90      -0.20f,-0.25f, 0.0f,
91      -0.20f,0.45f, 0.0f,
92
93      //Inferior _
94      -0.35f,-0.25f, 0.0f,
95      0.18f,-0.25f, 0.0f,
96      -0.35f,-0.11f, 0.0f,
97
98      -0.35f,-0.11f, 0.0f,
99      0.18f,-0.11f, 0.0f,
100     0.18f,-0.25f, 0.0f,
101
102     //----- LETRA C -----
103     //Superior
104     0.30f,-0.25f, 0.0f,
105     0.85f,-0.25f, 0.0f,
106     0.85f,-0.40f, 0.0f,
107
108     0.85f,-0.40f, 0.0f,
109     0.30f,-0.40f, 0.0f,
110     0.30f,-0.25f, 0.0f,
111
112     //Vertical
113     0.30f,-0.40f, 0.0f,
114     0.30f,-0.90f, 0.0f,
115     0.45f,-0.90f, 0.0f,
116
117     0.45f,-0.90f, 0.0f,
118     0.45f,-0.40f, 0.0f,
119     0.30f,-0.40f, 0.0f,
120
121     //Inferior
122     0.30f,-0.90f, 0.0f,
123     0.85f,-0.90f, 0.0f,
124     0.30f,-0.75f, 0.0f,
125
126     0.30f,-0.75f, 0.0f,
127     0.85f,-0.75f, 0.0f,
128     0.85f,-0.90f, 0.0f,
129
130 };
131 glGenVertexArrays(1, &VAO); //generar 1 VAO
132 glBindVertexArray(VAO); //asignar VAO

```

```

134     glGenBuffers(1, &VBO);
135     glBindBuffer(GL_ARRAY_BUFFER, VBO);
136     glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW); //pasar l
137
138     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(GL_FLOAT), (GLvoid*)0);/
139     glEnableVertexAttribArray(0);
140     //agregar valores a vértices y luego declarar un nuevo vertexAttribPointer
141     glBindBuffer(GL_ARRAY_BUFFER, 0);
142     glBindVertexArray(0);
143
144 }
145 void AddShader(GLuint theProgram, const char* shaderCode, GLenum shaderType) //Función para
146
147 //the Program recibe los datos de theShader
148
149 {
150     GLuint theShader = glCreateShader(shaderType); //theShader es un shader que se crea de a
151     const GLchar* theCode[1];
152     theCode[0] = shaderCode; //shaderCode es el texto que se le pasa a theCode
153     GLint codeLength[1];
154     codeLength[0] = strlen(shaderCode); //longitud del texto
155     glShaderSource(theShader, 1, theCode, codeLength); //Se le asigna al shader el código
156     glCompileShader(theShader); //Se compila el shader
157     GLint result = 0;
158     GLchar eLog[1024] = { 0 };
159     //verificaciones y prevención de errores
160     glGetShaderiv(theShader, GL_COMPILE_STATUS, &result);
161     if (!result)
162     {
163         glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
164         printf("EL error al compilar el shader %d es: %s \n", shaderType, eLog);
165         return;
166     }
167     glAttachShader(theProgram, theShader); //Si no hubo problemas se asigna el shader a theP

170 void CompileShaders() {
171     shader = glCreateProgram(); //se crea un programa
172     if (!shader)
173     {
174         printf("Error creando el shader");
175         return;
176     }
177     AddShader(shader, vShader, GL_VERTEX_SHADER); //Agregar vertex shader
178     AddShader(shader, fShader, GL_FRAGMENT_SHADER); //Agregar fragment shader
179     //Para terminar de linkear el programa y ver que no tengamos errores
180     GLint result = 0;
181     GLchar eLog[1024] = { 0 };
182     glLinkProgram(shader); //se linkean los shaders a la tarjeta gráfica
183     //verificaciones y prevención de errores
184     glGetProgramiv(shader, GL_LINK_STATUS, &result);
185     if (!result)
186     {
187         glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
188         printf("EL error al linkear es: %s \n", eLog);
189         return;
190     }
191     glValidateProgram(shader);
192     glGetProgramiv(shader, GL_VALIDATE_STATUS, &result);
193     if (!result)
194     {
195         glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
196         printf("EL error al validar es: %s \n", eLog);
197         return;
198     }

```

```

203     int main()
204     {
205         //Iniciación de GLFW
206         if (!glfwInit())
207         {
208             printf("Falló inicializar GLFW");
209             glfwTerminate();
210             return 1;
211         }
212
213         //**** LAS SIGUIENTES 4 LÍNEAS SE COMENTAN EN DADO CASO DE QUE AL USUARIO NO LE FUNCION
214
215         //Asignando variables de GLFW y propiedades de ventana
216         glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 4);
217         glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
218         //para solo usar el core profile de OpenGL y no tener retrocompatibilidad
219         glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
220         glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
221
222         //CREAR VENTANA
223         GLFWwindow *mainWindow = glfwCreateWindow(WIDTH, HEIGHT, "Primer ventana", NULL, NULL);
224
225         if (!mainWindow)
226         {
227             printf("Fallo en crearse la ventana con GLFW");
228             glfwTerminate();
229             return 1;
230         }
231         //Obtener tamaño de Buffer
232         int BufferWidth, BufferHeight;
233         glfwGetFramebufferSize(mainWindow, &BufferWidth, &BufferHeight);
234
235         //asignar el contexto
236         glfwMakeContextCurrent(mainWindow);

```

```

238         //permitir nuevas extensiones
239         glewExperimental = GL_TRUE;
240
241         if (glewInit() != GLEW_OK)
242         {
243             printf("Falló inicialización de GLEW");
244             glfwDestroyWindow(mainWindow);
245             glfwTerminate();
246             return 1;
247         }
248
249         // Asignar valores de la ventana y coordenadas
250         //Asignar Viewport
251         glViewport(0, 0, BufferWidth, BufferHeight);
252
253         //Llamada a las funciones creadas antes del main
254         CrearTriangulo();
255         CompileShaders();
256
257         // ----- Inicialización de random -----
258         srand(static_cast<unsigned int>(time(0)));
259
260         // Generar un primer color aleatorio inicial
261         float r = static_cast<float>(rand()) / RAND_MAX;
262         float g = static_cast<float>(rand()) / RAND_MAX;
263         float b = static_cast<float>(rand()) / RAND_MAX;
264
265         double lastTime = glfwGetTime();
266
267
268         //Loop mientras no se cierra la ventana
269         while (!glfwWindowShouldClose(mainWindow))
270         {
271             //Recibir eventos del usuario
272             glfwPollEvents();

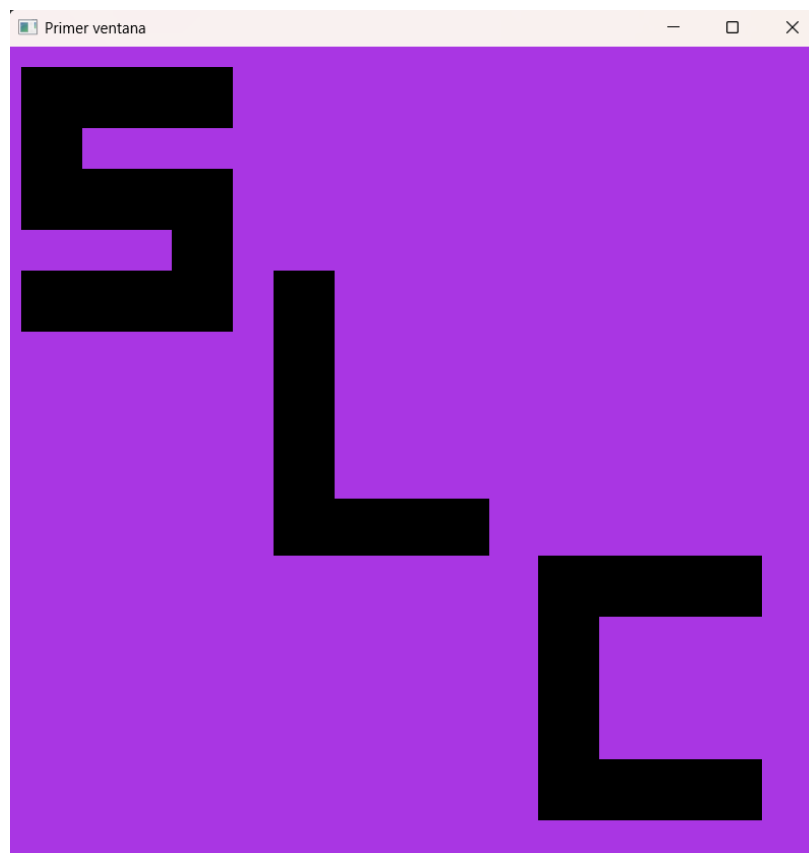
```

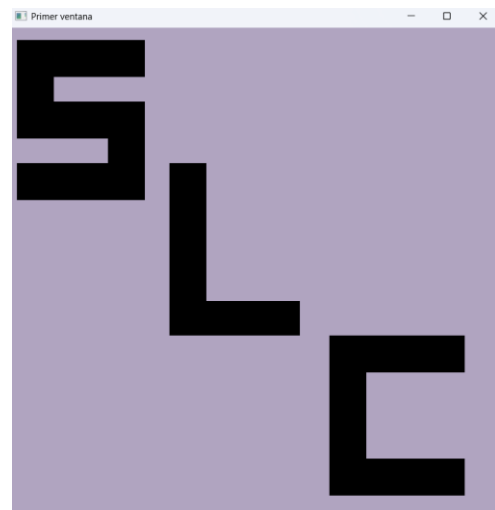
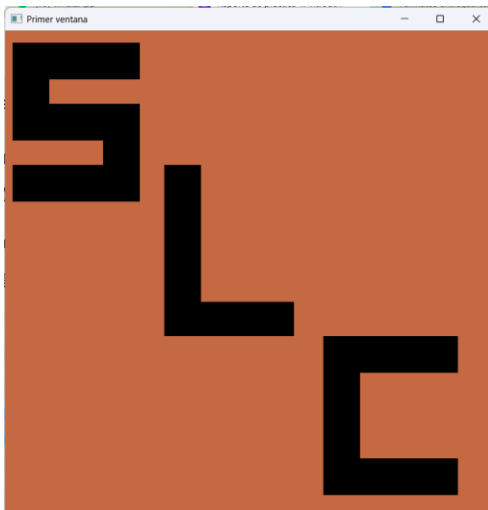
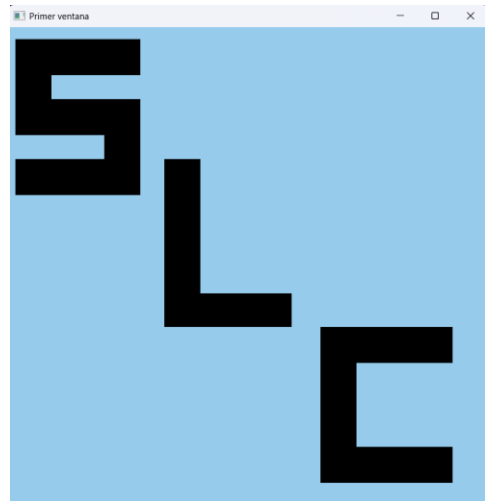
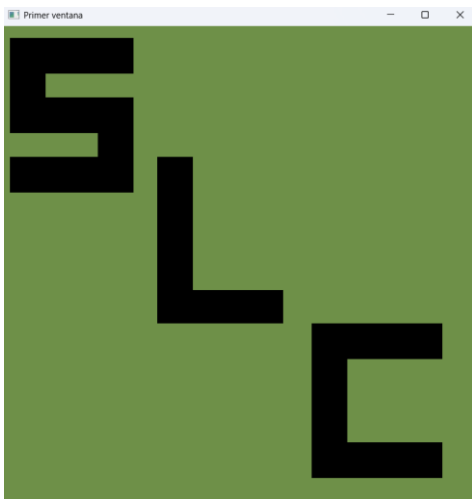
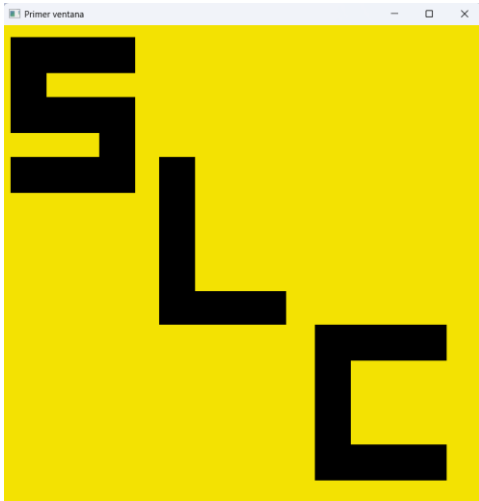
```

274 // -----> Cada 2 segundos cambia el color
275 double currentTime = glfwGetTime();
276 if (currentTime - lastTime >= 2.0) {
277     r = static_cast<float>(rand()) / RAND_MAX;
278     g = static_cast<float>(rand()) / RAND_MAX;
279     b = static_cast<float>(rand()) / RAND_MAX;
280     lastTime = currentTime;
281 }
282
283 //Limpiar la ventana
284 glClearColor(r, g, b, 1.0f); // Fondo random
285 glClear(GL_COLOR_BUFFER_BIT);
286
287 glUseProgram(shader);
288
289 glBindVertexArray(VAO);
290 glDrawArrays(GL_TRIANGLES, 0, 80);
291 glBindVertexArray(0);
292
293 glUseProgram(0);
294
295 glfwSwapBuffers(mainWindow);
296
297 //NO ESCRIBIR NINGUNA LÍNEA DESPUÉS DE glfwSwapBuffers(mainWindow);
298 }
299
300 return 0;
301 }
302

```

Los ejecutables y nuestros resultados son estos:





### **Problemas presentados:**

Tuve un poco de problemas al usar el random para que salieran diferentes colores en la pantalla, inicie el fondo en color negro, entonces siempre iniciaba así y tuve que agregar dos random, uno para la inicialización y después para que cada 2 segundos se cambiara todo.

También tuve problemas para las coordenadas, pero solo con la práctica pude realizar entender todo mucho mejor.

### **Conclusión:**

- a. Los ejercicios del reporte: No fueron complicados estos ejercicios, sé que vamos empezando y esto nos está ayudando a entender los temas desde cero, entender las coordenadas y que aun así debemos tener las bases de programación para realizarlos al cien.
- b. Comentarios generales: Sentí que la explicación fue muy completa, el tiempo en que se realizó y toda la clase como tal me parece muy bien ordenado y organizado.
- c. Conclusión: La práctica fue “sencilla” un poco laboriosa pero sencilla para realizar, se me está haciendo más sencillo debido a que ya llevé esta materia, pero ahora si estoy entendiendo mucho mejor que antes.

### **Bibliografía en formato APA**

-TylerMSFT. (2023, June 16). . Microsoft.com. <https://learn.microsoft.com/es-es/cpp/standard-library/cstdlib?view=msvc-170>

-TylerMSFT. (2024, August 4). . Microsoft.com. <https://learn.microsoft.com/es-es/cpp/standard-library/ctime?view=msvc-170>